

• **Interfaccia**: insieme di chiamate funzione ~ qua si lavora a livello trasporto (TCP e UDP)

(livello + basso per fare programmazione di rete)

• **host**: è una macchina (hardware) , ha una sua interfaccia di rete attiva

l'ip ti differenzia
l'interfaccia di rete
invece la porta
il processo
esso

• **processo**: programma in esecuzione ~ legge, trasmette, riceve pacchetti ~ rappresentato da una porta

• **applicazione**: insieme di processi. Per esempio il web

Host: browser
su mio PC

Host server: Apache (processo esecuzione sulla macchina remota)

• Per scoprire l'ip Linux: `ifconfig - a`

Windows: `ipconfig /all`

• Noi trasmettiamo gruppi di byte trasmessi in pacchetto, pdu, datagram

• **applicazione orientata al datagram**: quando ogni pacchetto pensa a se stesso.

• **applicazione orientata alla connessione**: tra i pacchetti c'è una relazione: parte di un messaggio più grande (ad es.

un'immagine) ~ concetto della connessione (che sarebbe una sessione)

→ Pattern "modo di scrivere"

• **Modello client/server**: hai 2 ruoli e il client fa la prima iterazione il server dà la risposta.

NB: client e server sono processi.

L'insieme di client e server forma l'applicazione di rete

• **interfaccia socket**: strumento che permette di usare i servizi di rete

→ UDP
 ↗ send
 ↘ receive

Domanda da esame:

• Perché con una send in un colpo solo non puoi mandare 2 pacchetti? Perché il protocollo UDP non può frammentare e quindi il ricevente non sa come agire

local host: 127.0.0.1

• Puoi fare un'applicazione affidabile con UDP? Sì: se ti implementi tu gli ACK

• **TCP**: ovviamente anche con il TCP i dati si perdono ma i dati si ritrasmettono

qua 4 primitive:

- 1) aprire connessione
- 2) trasmettere
- 3) ricevere
- 4) chiudere connessione (sessione)

• Temi d'esame:

- 1) UDP , 2) TCP , 3) UDP , 4) TCP

